P-Grafo: formato para GitHub Projects

Objetivo

Este documento define un formato práctico para gestionar P-Grafo en **GitHub Projects (new)** con metodología **Kanban**, usando Issues como unidad principal de trabajo y un tablero agrupado por `Status`.[cite:6268][cite:6272][cite:6273]

GitHub Projects permite organizar trabajo con vistas de tablero y tabla, columnas basadas en un campo `Status`, y campos personalizados como `Priority` y `Estimate`, lo que encaja bien con un flujo Kanban orientado a visualizar trabajo, limitar WIP y mejorar continuamente el flujo.[cite:6253][cite:6255][cite:6259][cite:6272]

Modelo recomendado

Unidad de trabajo

- Usar **GitHub Issues** para tareas reales.
- Usar **Draft items** solo para ideas muy tempranas o trabajo aún no refinado.
- Vincular Pull Requests a las Issues para trazabilidad.

Vistas del proyecto

- **Board por Status**: vista principal Kanban.[cite:6268][cite:6276]
- **Table completa**: edición masiva de campos.[cite:6272][cite:6276]
- **Board por Priority**: foco de trabajo.[cite:6272]
- **Vista Blocked**: filtro `Status = Blocked`.
- **Vista MVP**: filtro `Target = MVP`.

Campos personalizados recomendados

Crear estos campos en GitHub Projects:

Campo	Tipo	Valores recomendados
`Status`	Single select	`Backlog`, `Ready`, `In Progress`, `Review`, `Done`, `Blocked`
`Priority`	Single select	`P0`, `P1`, `P2`, `P3`
`Type`	Single select	`Epic`, `Feature`, `Task`, `Spike`, `Bug`, `Tech Debt`
`Area`	Single select	`Architecture`, `Domain`, `Application`, `Infrastructure`, `CLI`, `Science`, `Testing`, `Docs`, `DevEx`
`Estimate`	Number	`1`, `2`, `3`, `5`, `8`
`Risk`	Single select	`Low`, `Medium`, `High`
`Target`	Single select	`Foundation`, `MVP`, `Post-MVP`

GitHub documenta el uso de campos como `Status`, `Priority`, `Estimate`, `Type` y otros metadatos para organizar y personalizar la gestión del trabajo en Projects.[cite:6272][cite:6275]

Flujo Kanban recomendado

Columnas del tablero

- `Backlog`: trabajo identificado pero aún no preparado.
- `Ready`: trabajo refinado y listo para empezar.
- `In Progress`: trabajo activo.
- `Review`: trabajo terminado pendiente de revisión técnica o funcional.

- `Done`: trabajo aceptado.
- `Blocked`: trabajo detenido por dependencia o incidencia.

Límites WIP iniciales

- `In Progress`: 2
- `Review`: 2

Kanban se apoya en visualizar el trabajo, limitar el trabajo en curso y gestionar el flujo de forma explícita; por eso conviene definir pocas columnas, políticas claras y límites WIP desde el principio.[cite:6253][cite:6255][cite:6260][cite:6262]

Formato estándar para una Issue

Usar esta plantilla para todas las tareas:

```
## Contexto
Breve explicación de por qué existe esta tarea.

## Objetivo
Resultado concreto que debe quedar hecho.

## Alcance
- Punto 1
- Punto 2
- Punto 3

## Criterios de aceptación
- [ ] Criterio verificable 1
- [ ] Criterio verificable 2
- [ ] Criterio verificable 3

## Dependencias
- #123
- Ninguna

## Notas técnicas
Observaciones de implementación, restricciones o decisiones.
```

Este formato ayuda a que cada tarjeta tenga contexto, criterio de terminado y dependencia explícita, que son prácticas coherentes con la gestión visual y basada en flujo de Kanban.[cite:6253][cite:6256][cite:6259]

Convención de títulos

Usar prefijos consistentes para mejorar filtrado y lectura rápida:

- `ARCH-001 Crear C4 contexto`
- `ARCH-002 Crear C4 contenedores`
- `DEVX-001 Crear pyproject.toml`
- `DOM-001 Implementar ProteinSequence`
- `APP-001 Crear RunPredictionUseCase`
- `CLI-001 Crear comando predict`

Formato fuente para preparar tareas

Esta tabla sirve como fuente base antes de crear las issues:

Title	Body	Labels	Status	Priority	Type	Area	Estimate	Risk	Targe
Crear repositorio GitHub `p-grafo`	Ver plantilla estándar.	`setup`, `github`, `architecture`	`Backlog`	`P0`	`Task`	`DevEx`	`1`	`Low`	`Founda`

Title	Body	Labels	Status	Priority	Type	Area	Estimate	Risk	Targe
Crear `pyproject.toml`	Ver plantilla estándar.	`python`, `build`	Backlog	P0	Task	DevEx	1	Low	Founda
Crear estructura `src/p_grafo`	Ver plantilla estándar.	`architecture`, `python`	Backlog	P0	Task	Architecture	2	Low	Founda
Configurar `pytest`	Ver plantilla estándar.	`testing`, `python`	Backlog	P1	Task	Testing	1	Low	Founda
Configurar `ruff`	Ver plantilla estándar.	`quality`, `python`	Backlog	P1	Task	DevEx	1	Low	Founda
Crear workflow CI	Ver plantilla estándar.	`ci`, `github`	Backlog	P0	Task	DevEx	2	Medium	Founda
Redactar README inicial	Ver plantilla estándar.	`docs`, `github`	Backlog	P1	Task	Docs	2	Low	Founda
Crear C4 contexto	Ver plantilla estándar.	`docs`, `architecture`	Backlog	P0	Task	Architecture	2	Low	Founda
Crear C4 contenedores	Ver plantilla estándar.	`docs`, `architecture`	Backlog	P0	Task	Architecture	2	Low	Founda
Implementar `ProteinSequence`	Ver plantilla estándar.	`domain`, `science`	Backlog	P0	Feature	Domain	3	Medium	MVP

Plantilla CSV para importación de Issues

GitHub Projects no ofrece una importación CSV universal y directa al tablero como flujo principal, pero es común crear issues en lote mediante CSV/JSON usando acciones o importadores y luego añadirlas al proyecto.[cite:6270] [cite:6271][cite:6274]

Formato base sugerido:

```
title,body,labels,assignees,milestone
"Crear repositorio GitHub p-grafo","## Contexto
Necesitamos el repositorio base del proyecto.

## Objetivo
Disponer de un repo inicial con configuración mínima.

## Alcance
- Crear repo
- Configurar visibilidad
- Añadir rama principal protegida

## Criterios de aceptación
- [ ] El repositorio existe
- [ ] La rama principal está protegida
- [ ] El README inicial está creado

## Dependencias
- Ninguna

## Notas técnicas
Nombre propuesto: p-grafo","setup;github;architecture",,Foundation
"Crear pyproject.toml","## Contexto
Se necesita inicializar el proyecto Python.

## Objetivo
Definir metadatos, dependencias y configuración base.
```

```
## Alcance
- Crear pyproject
- Definir paquete
- Configurar herramientas

## Criterios de aceptación
- [ ] El archivo existe
- [ ] El proyecto instala correctamente
- [ ] pytest y lint están declarados

## Dependencias
- Ninguna

## Notas técnicas
Evaluar uv o poetry","python;build",,Foundation
```

Automatizaciones mínimas

Configurar estas automatizaciones del proyecto:

- Al añadir un item al proyecto, asignar `Status = Backlog`.
- Al cerrar una issue, mover a `Done`.
- Al reabrir una issue, devolver a `Backlog` o `Ready`, según la política definida por el equipo.[cite:6273][cite:6276]

Políticas operativas

Ready

Una tarjeta entra en `Ready` cuando:

- Tiene descripción clara.
- Tiene criterios de aceptación verificables.
- Tiene dependencias identificadas.
- Su tamaño es pequeño, idealmente resoluble en un día o menos.

Review

Una tarjeta entra en `Review` cuando:

- El código está subido.
- Los tests relevantes pasan.
- La documentación mínima está actualizada.
- La issue referencia su Pull Request.

Done

Una tarjeta entra en `Done` cuando:

- Ha sido revisada.
- Cumple criterios de aceptación.
- No quedan bloqueos abiertos.
- Está integrada en la rama principal o equivalente.

Configuración mínima recomendada

- **Campos**: `Status`, `Priority`, `Type`, `Area`, `Estimate`, `Risk`, `Target`.
- **Vista principal**: board agrupado por `Status`.[cite:6268]
- **Vista operativa**: tabla con edición masiva de campos.[cite:6272]
- **Vista de foco**: agrupación por `Priority`.[cite:6272]

- **Métricas**: lead time, cycle time y throughput, que son métricas habituales para vigilar flujo en Kanban. [cite:6255][cite:6259]

Recomendación práctica para P-Grafo

El camino más simple es:

1. Crear las labels del repositorio.
2. Crear el Project y sus campos personalizados.
3. Crear las primeras 10 a 20 Issues manualmente o mediante importador.
4. Añadirlas al Project.
5. Agrupar la vista principal por `Status`.
6. Empezar con pocas tarjetas en `Ready` para no saturar `In Progress`.

Ese enfoque encaja con la idea de Kanban de empezar con un flujo visible, limitar el WIP y ajustar políticas conforme aparecen cuellos de botella reales.[cite:6253][cite:6255][cite:6260]